

**OBJECT ORIENTED PROGRAMMING WITH JAVA
LABORATORY**

Subject code: BCS306A

[As per Choice Based Credit System (CBCS) scheme]

(Effective from the academic year 2022 -2023)

OBJECT ORIENTED PROGRAMMING WITH JAVA LABORATORY

Subject Code : BCS306A

IAMarks: 50

Number of Lecture Hours/Week : 2T+ 2P

ExamMarks: 50

1. Develop a JAVA program to add TWO matrices of suitable order N (The value of N should be read from command line arguments).
2. Develop a stack class to hold a maximum of 10 integers with suitable methods. Develop a JAVA main method to illustrate Stack operations.
3. A class called Employee, which models an employee with an ID, name and salary, is designed as shown in the following class diagram. The method raise Salary (percent) increases the salary by the given percentage. Develop the Employee class and suitable main method for demonstration.
4. A class called MyPoint, which models a 2D point with x and y coordinates, is designed as follows:
 - Two instance variables x (int) and y (int).
 - A default (or "no-arg") constructor that construct a point at the default location of (0, 0).
 - A overloaded constructor that constructs a point with the given x and y coordinates.
 - A method setXY() to set both x and y.
 - A method getXY() which returns the x and y in a 2-element int array.
 - A toString() method that returns a string description of the instance in the format "(x, y)".
 - A method called distance(int x, int y) that returns the distance from this point to another point at the given (x, y) coordinates
 - An overloaded distance(MyPoint another) that returns the distance from this point to the given MyPoint instance (called another)
 - Another overloaded distance() method that returns the distance from this point to the origin (0,0)

Develop the code for the class MyPoint. Also develop a JAVA program (called TestMyPoint) to test all the methods defined in the class.

5. Develop a JAVA program to create a class named shape. Create three sub classes namely: circle, triangle and square, each class has two member functions named draw () and erase (). Demonstrate polymorphism concepts by developing suitable methods, defining member data and main program.
6. Develop a JAVA program to create an abstract class Shape with abstract methods calculateArea() and calculate Perimeter(). Create subclasses Circle and Triangle that extend the Shape class and implement the respective methods to calculate the area and perimeter of each shape.

7. Develop a JAVA program to create an interface Resizable with methods `resizeWidth(int width)` and `resizeHeight(int height)` that allow an object to be resized. Create a class `Rectangle` that implements the `Resizable` interface and implements the `resize` methods
8. Develop a JAVA program to create an outer class with a function `display`. Create another class inside the outer class named `inner` with a function called `display` and call the two functions in the main class.
9. Develop a JAVA program to raise a custom exception (user defined exception) for `DivisionByZero` using `try`, `catch`, `throw` and `finally`.
10. Develop a JAVA program to create a package named `mypack` and import & implement it in a suitable class.
11. Write a program to illustrate creation of threads using `Runnable` class. (start method start each of the newly created thread. Inside the `run` method there is `sleep()` for suspend the thread for 500 milliseconds).
12. Develop a program to create a class `MyThread` in this class a constructor, call the base class constructor, using `super` and start the thread. The `run` method of the class starts after this. It can be observed that both main thread and created child thread are executed concurrently.

1. Develop a JAVA program to add TWO matrices of suitable order N (The value of N should be read from command line arguments).

Add.java

```
import java.util.Scanner;
public class Add
{
    public static void main(String[] args)
    {
        int m, n; //Declare matrix size
        Scanner scan = new Scanner(System.in);
        System.out.print("Enter the number of rows in the matrix:");
        m = scan.nextInt(); //Initialize first matrix size
        System.out.print("Enter the number of columns in the matrix:");
        n = scan.nextInt(); //Initialize second matrix size

        int a[][] = new int[m][n]; //Declare first matrix
        int b[][] = new int[m][n]; //Declare second matrix
        int c[][] = new int[m][n]; //Declare third matrix
        //Initialize the first matrix
        System.out.println("Enter all the elements of first matrix:");
        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < n; j++)
            {
                a[i][j] = scan.nextInt();
            }
        }
        System.out.println("");
        //Initialize the second matrix
        System.out.println("Enter all the elements of second matrix:");
        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < n; j++)
            {
                b[i][j] = scan.nextInt();
            }
        }
    }
}
```

```
//Loop to add matrix elements
for (int i = 0; i < m; i++)
{
    for (int j = 0; j < n; j++)
    {

        c[i][j] = a[i][j] + b[i][j];

    }
}
//Print the resultant matrix
System.out.println("Matrix after addition:");
for (int i = 0; i < m; i++)
{
    for (int j = 0; j < n; j++)
    {
        System.out.print(c[i][j]+" ");
    }
    System.out.println("");
}
}
```

OUTPUT:

Enter the number of rows in the matrix:3
Enter the number of columns in the matrix:3
Enter all the elements of first matrix:
1 2 3
4 5 6
7 8 9

Enter all the elements of second matrix:
9 8 7
6 5 4
3 2 1
Matrix after addition:
10 10 10
10 10 10
10 10 10

2. Develop a stack class to hold a maximum of 10 integers with suitable methods. Develop a JAVA main method to illustrate Stack operations.

Stack.java

```
import java.util.*;
public class Stack
{
    int s[]=new int[10]; int top= -1;
    int size=3;

    void push(int i)
    {
        if(top==size-1)
            System.out.println("Stack Overflow");

        else
        {
            s[++top] = i;
        }
    }

    void pop( )
    {
        if (top == -1)
        {
            System.out.println("Stack Underflow");
        }
        else
        {
            System.out.println(" Popped Element= " + s[top]);
            top--;
        }
    }

    void display( )
    {
        if(top == -1)
        {
            System.out.println("Stack is Empty\n");
        }

        else
        {
            System.out.println("Stack Elements are:\n");
            for (int i = top; i >= 0; i--)
                System.out.println(s[i]);
        }
    }
}
```

```
public static void main(String args[])
{
    Scanner scan = new Scanner(System.in);
    Stack stk = new Stack();
    for(;;)
    {
        System.out.println("\n---Stack Operations---");
        System.out.println("1. Push");
        System.out.println("2. Pop");
        System.out.println("3. Display");
        System.out.println("4. Exit");

        System.out.println("Enter your choice:\n");
        int choice = scan.nextInt();
        switch (choice)
        {
            case 1 :
                System.out.println("Enter the element
to push");
                stk.push(scan.nextInt());
                break;

            case 2 :   stk.pop();
                break;

            case 3 :   stk.display();
                break;

            case 4 :   System.exit(0);

            default :
                System.out.println("Invalid Choice\n");
                break;
        }
    }
}
```

OUTPUT:

-----Stack Operations-----

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 3

Stack is Empty

-----Stack Operations-----

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 2

Stack Underflow

-----Stack Operations-----

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 1

Enter the element to push 10

-----Stack Operations-----

1. Push
2. Pop
3. Display

4. Exit

Enter your choice: 1

Enter the element to push 60

-----Stack Operations-----

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 1

Enter the element to push 30

-----Stack Operations-----

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 1

Enter the element to push 50

Stack Overflow

-----Stack Operations-----

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 3

Stack Elements are:

30 60 10

-----Stack Operations-----

1. Push

2. Pop

3. Display

4. Exit

Enter your choice: 2

Popped Element= 30

-----Stack Operations-----

1. Push

2. Pop

3. Display

4. Exit

Enter your choice: 3

Stack Elements are:

60 10

-----Stack Operations-----

1. Push

2. Pop

3. Display

4. Exit

Enter your choice: 4

2. A class called Employee, which models an employee with an ID, name and salary, is designed as shown in the following class diagram. The method raiseSalary (percent) increases the salary by the given percentage. Develop the Employee class and suitable main method for demonstration.

Employee.java

```
import java.util.Scanner;
public class Employee
{
    int id;
    String name;
    double salary;

    public Employee(int id, String name, double salary)
    {
        this.name = name;
        this.id = id;
        this.salary = salary;
    }

    public void display()
    {
        System.out.println("Id: " + id);
        System.out.println("Name: " + name);
        System.out.println("Salary: " + salary);
    }

    public void raiseSalary(double percentage)
    {
        salary = salary + (salary * percentage / 100);
    }

    public static void main(String[] args)
    {
        int p;
        Employee e1 = new Employee(8, "Rakesh", 2500);

        e1.display();
        Scanner scan= new Scanner(System.in);

        System.out.println("\nEnter the percentage to raise the salary");
        p=scan.nextInt();
        e1.raiseSalary(p);

        System.out.println("\nAfter raising salary");
        e1.display();
    }
}
```

OUTPUT:

Id: 8
Name: Rakesh
Salary: 2500.0

Enter the percentage to raise the salary
5

After raising salary
Id: 8
Name: Rakesh
Salary: 2625.0

3. A class called MyPoint, which models a 2D point with x and y coordinates, is designed as follows:

- ❖ Two instance variables x (int) and y (int).
- ❖ A default (or "no-arg") constructor that constructs a point at the default location of (0, 0).
- ❖ A overloaded constructor that constructs a point with the given x and y coordinates.
- ❖ A method setXY() to set both x and y.
- ❖ A method getXY() which returns the x and y in a 2-element int array.
- ❖ A toString() method that returns a string description of the instance in the format "(x, y)".
- ❖ A method called distance(int x, int y) that returns the distance from this point to another point at the given (x, y) coordinates
- ❖ An overloaded distance(MyPoint another) that returns the distance from this point to the given MyPoint instance (called another)
- ❖ Another overloaded distance() method that returns the distance from this point to the origin (0,0) Develop the code for the class MyPoint. Also develop a JAVA program (called TestMyPoint) to test all the methods defined in the class.

MyPoint.java

```
public class MyPoint
{
    private int x = 0;
    private int y = 0;

    public MyPoint()
    {
        this.x = 0;
        this.y = 0;
    }

    public MyPoint(int x, int y)
    {
        this.x = x;
        this.y = y;
    }

    public double distance(int x, int y)
    {
        int xDiff = this.x - x;
        int yDiff = this.y - y;
        return Math.sqrt(xDiff*xDiff + yDiff*yDiff);
    }

    public double distance(MyPoint another)
    {
        int xDiff = this.x - another.x;
        int yDiff = this.y - another.y;
        return Math.sqrt(xDiff*xDiff + yDiff*yDiff);
    }

    // Overloaded method to calculate distance to the origin (0,0)
    public double distance()
    {
        return Math.sqrt(x * x + y * y);
    }
}
```

```
    }

    public int getX()
    {
        return x;
    }

    public void setX(int x)
    {
        this.x = x;
    }

    public int getY()
    {
        return y;
    }

    public void setY(int y)
    {
        this.y = y;
    }

    public void setXY(int x, int y)
    {
        this.x = x;
        this.y = y;
    }

    public String toString()
    {
        return "(" + x + ", " + y + ")";
    }

    public static void main(String[] args)
    {
        // Creating MyPoint instances
        MyPoint point1 = new MyPoint(3, 4);
        MyPoint point2 = new MyPoint(6, 8);

        System.out.println("Point 1 coordinates: " +
point1.toString());
        System.out.println("Point 2 coordinates: " +
point2.toString());

        System.out.println("Distance from Point 1 to (6, 8): " +
point1.distance(6, 8));
        System.out.println("Distance from Point 1 to Point 2: " +
point1.distance(point2));
        System.out.println("Distance from Point 1 to the origin: " +
point1.distance());
    }
}
```


OUTPUT:

Point 1 coordinates: (3, 4)

Point 2 coordinates: (6, 8)

Distance from Point 1 to (6, 8): 5.0

Distance from Point 1 to Point 2: 5.0

Distance from Point 1 to the origin: 5.0

4. Develop a JAVA program to create a class named shape. Create three sub classes namely: circle, triangle and square, each class has two member functions named draw () and erase (). Demonstrate polymorphism concepts by developing suitable methods, defining member data and main program.

Shape.java

```
public class Shape
{
    // Member functions
    public void draw()
    {
        System.out.println("Drawing a shape");
    }

    public void erase()
    {
        System.out.println("Erasing a shape");
    }
}

// Circle class, a subclass of Shape
class Circle extends Shape
{
    public void draw()
    {
        System.out.println("Drawing a circle");
    }
    public void erase()
    {
        System.out.println("Erasing a circle");
    }
}

// Triangle class, a subclass of Shape
class Triangle extends Shape
{
    public void draw()
    {
        System.out.println("Drawing a triangle");
    }

    public void erase()
    {
        System.out.println("Erasing a triangle");
    }
}

// Square class, a subclass of Shape
class Square extends Shape
{
```

```
public void draw()
{
    System.out.println("Drawing a square");
}

public void erase()
{
    System.out.println("Erasing a square");
}
}

// Main program to demonstrate polymorphism
class Main
{
    public static void main(String[] args)
    {
        // Creating objects of different shapes
        Circle c = new Circle();
        Triangle t = new Triangle();
        Square s = new Square();

        // Demonstrating polymorphism by calling draw and erase methods
        System.out.println("Using Circle object:");
        c.draw();
        c.erase();

        System.out.println("\nUsing Triangle object:");
        t.draw();
        t.erase();

        System.out.println("\nUsing Square object:");
        s.draw();
        s.erase();
    }
}
```

OUTPUT:

Using Circle object:
Drawing a circle
Erasing a circle

Using Triangle object:
Drawing a triangle
Erasing a triangle

Using Square object:
Drawing a square
Erasing a square

5. Develop a JAVA program to create an abstract class Shape with abstract methods calculateArea() and calculatePerimeter(). Create subclasses Circle and Triangle that extend the Shape class and implement the respective methods to calculate the area and perimeter of each shape.

Shape.java

```
public abstract class Shape
{
    // Abstract methods to calculate area and perimeter
    public abstract double calculateArea();
    public abstract double calculatePerimeter();
}

// Circle class, a subclass of Shape
class Circle extends Shape
{
    private double radius;

    public Circle(double radius)
    {
        this.radius = radius;
    }

    // Implementing abstract methods
    public double calculateArea()
    {
        return Math.PI * Math.pow(radius, 2);
    }

    public double calculatePerimeter()
    {
        return 2 * Math.PI * radius;
    }
}

class Triangle extends Shape
{
    private double side1, side2, side3;

    // Constructor
    public Triangle(double side1, double side2, double side3)
    {
        this.side1 = side1;
        this.side2 = side2;
        this.side3 = side3;
    }

    // Implementing abstract methods
    public double calculateArea()
    {
        // Using Heron's formula to calculate the area of a triangle
        double s = (side1 + side2 + side3) / 2;
```

```
        return Math.sqrt(s * (s - side1) * (s - side2) * (s - side3));
    }

    public double calculatePerimeter()
    {
        return side1 + side2 + side3;
    }
}

class Main {
    public static void main(String[] args) {
        // Creating objects of Circle and Triangle
        Circle circle = new Circle(5.0);
        Triangle triangle = new Triangle(3.0, 4.0, 5.0);

        // Displaying area and perimeter of the Circle
        System.out.println("Circle - Area: " + circle.calculateArea());
        System.out.println("Circle - Perimeter: " +
circle.calculatePerimeter());

        // Displaying area and perimeter of the Triangle
        System.out.println("Triangle - Area: " +
triangle.calculateArea());
        System.out.println("Triangle - Perimeter: " +
triangle.calculatePerimeter());
    }
}
```

OUTPUT:

Circle - Area: 78.53981633974483
Circle - Perimeter: 31.41592653589793
Triangle - Area: 6.0
Triangle - Perimeter: 12.0

6. Develop a JAVA program to create an interface Resizable with methods `resizeWidth(int width)` and `resizeHeight(int height)` that allow an object to be resized. Create a class `Rectangle` that implements the `Resizable` interface and implements the resize methods.

Rectangle.java

```
interface Resizable
{
    void resizeWidth(int width);
    void resizeHeight(int height);
}

class Rectangle implements Resizable
{
    private int width;
    private int height;

    public Rectangle(int width, int height)
    {
        this.width = width;
        this.height = height;
    }

    public void resizeWidth(int newWidth)
    {
        this.width = newWidth;
    }
    public void resizeHeight(int newHeight)
    {
        this.height = newHeight;
    }

    public void display()
    {
        System.out.println("Rectangle width: " + width + ", height: " + height);
    }

    public static void main(String[] args)
    {
        Rectangle rectangle = new Rectangle(5, 10);
        rectangle.display();

        rectangle.resizeWidth(8);
        rectangle.resizeHeight(15);
        rectangle.display();
    }
}
```

Output:

Rectangle width: 5, height: 10

Rectangle width: 8, height: 15

7. Develop a JAVA program to create an outer class with a function display. Create another class inside the outer class named inner with a function called display and call the two functions in the main class.

Outerclass.java

```
class Outerclass
{
    void display()
    {
        System.out.println("Outer class display method.");
    }

    class Innerclass
    {
        void display()
        {
            System.out.println("Inner class display method.");
        }
    }

    public static void main(String[] args)
    {
        Outerclass outer = new Outerclass();
        outer.display(); // Call the outer class's display() method

        Outerclass.Innerclass inner = outer.new Innerclass();
        inner.display(); // Call the inner class's display() method
    }
}
```

Output:

Outer class display() method.

Inner class display() method.

8. Develop a JAVA program to raise a custom exception (user defined exception) for DivisionByZero using try, catch, throw and finally.

```

                                CustomDivision.java
public class CustomDivision
{
    public static void main(String[] args) {
        int numerator = 10;
        int denominator = 0;

        try
        {
            if (denominator == 0)
            {
                throw new DivisionByZeroException("Division by zero is not allowed!");
            }
            int result = numerator / denominator;
            System.out.println("Result: " + result);
        }

        catch (DivisionByZeroException e)
        {
            System.err.println("Error: " + e.getMessage());
        }

        finally
        {
            System.out.println("This block always executes,
regardless of exceptions.");
        }
    }
}

// Custom exception class for DivisionByZero
class DivisionByZeroException extends Exception
{
    public DivisionByZeroException(String message)
    {
        super(message);
    }
}

```

OUTPUT:

This block always executes, regardless of exceptions.
 Error: Division by zero is not allowed!

9. Develop a JAVA program to create a package named mypack and import & implement it in a suitable class.

Package: mypack
Class name: MyClass

```
package mypack;  
  
public class MyClass  
{  
    public void display()  
    {  
        System.out.println("This is a method from the mypack package!");  
    }  
}
```

Example.java

```
import mypack.MyClass;  
public class Example  
{  
    public static void main(String[] args)  
    {  
        MyClass obj = new MyClass();  
        obj.display(); // Access the method from the imported package  
    }  
}
```

Output: Run Example.java

This is a method from the mypack package!

10. Write a program to illustrate creation of threads using runnable class. (start method start each of the newly created thread. Inside the run method there is sleep() for suspend the thread for 500 milliseconds).

Mythread.java

```
public class Mythread implements Runnable
{
    public void run()
    {
        for (int i = 1; i <= 5; i++)
        {
            System.out.println(Thread.currentThread().getName() + " i is " + i);
            try
            {
                //sleep current thread for 500 ms
                Thread.sleep(500);
            }
            catch (InterruptedException e)
            {
                //print the exception message if occurred
                System.out.println(e.getMessage());
            }
        }
    }
}

class ThreadExample
{
    public static void main(String[] args)
    {
        Mythread myThread = new Mythread();
        //thread 1
        Thread t1 = new Thread(myThread);

        //thread2
        Thread t2 = new Thread(myThread);

        //thread 3
        Thread t3 = new Thread(myThread);

        //starting all 3 threads now
        t1.start();
        t2.start();
        t3.start();
    }
}
```

Output:

```
Thread-0 i is 1
Thread-2 i is 1
Thread-1 i is 1
Thread-0 i is 2
```


Thread-1 i is 2
Thread-2 i is 2
Thread-0 i is 3
Thread-2 i is 3
Thread-1 i is 3
Thread-0 i is 4
Thread-1 i is 4
Thread-2 i is 4
Thread-0 i is 5
Thread-2 i is 5
Thread-1 i is 5

11. Develop a program to create a class MyThread in this class a constructor, call the base class constructor, using super and start the thread. The run method of the class starts after this. It can be observed that both main thread and created child thread are executed concurrently.

TestMyThread.java

```
public class TestMyThread
{
    public static void main(String args[])
    {
        new MyThread();
        try
        {
            for( int k = 5; k > 0; k--)
            {
                System.out.println ("Running main thread : " + k);
                Thread.sleep(1000);
            }
        }
        catch (InterruptedException e)
        {
        }
        System.out.println ("Exiting main thread . . .");
    }
}

class MyThread extends Thread
{
    MyThread()
    {
        super("Using Thread class");
        System.out.println("Child thread: " + this);
        start();
    }
    public void run()
    {
        try
        {
            for ( int i =5; i > 0; i--)
            {
                System.out.println("Child thread" + i);
                Thread.sleep (500);
            }
        }
        catch (InterruptedException e)
        {
        }
        System.out.println("exiting child thread ...");
    }
}
```

Output:

```
Child thread:Thread[Using Thread class,5,main]
Running main thread :5
Child thread5
Child thread4
Child thread3
Running main thread :4 Child
thread2
Running main thread :3 Child
thread1
exiting child thread ...
Running main thread :2
Running main thread :1
Exiting main thread . . .
```